SO-101 색 큐브 쌓기: 모방학습, 고전 파이프라인, 그리고 시뮬레이션→실물 전이

기간 2026-09-13 ~ 10-09 (27일) · **장비** SO-101 리더/팔로워 (Feetech STS3215), Mac mini M4 Pro, UVC 카메라 2대(탑·손목), 3 cm 색 큐브 · **소프트웨어** LeRobot, ACT, SmolVLA, OpenCV, MuJoCo · **학습** RunPod (RTX 4090 / 6000 Ada / 3090 / 5090) · **허브** `moonbeards`

> 한 줄 요약: 같은 로봇, 같은 작업(주황 큐브를 빨강 큐브 위에 쌓기)을 **모방학습**(ACT → SmolVLA + DAgger)과 **고전 파이프라인**(카메라 보정 → 역기구학 → 시각 서보)으로 각각 구현하고, 같은 잣대로 비교했다. 시연 범위 안에서는 고전 10/10 대 학습 4/10, 범위 밖에서는 7/10 대 0/10. 두 방식은 서로 다른 곳에서 무너진다. 이어서 MuJoCo 시뮬레이터를 실측으로 맞추고 **사람 시연 없이 자동 생성한 시연만으로** 학습한 정책을 실물에 올렸더니 2/10(실물 시연 정책 3~4/10과 같은 구간)을 냈다. 시뮬의 병목은 물리가 아니라 시연의 다양성이었다.
>
> **English abstract.** Same low-cost arm (SO-101), same task (stack an orange cube on a red one), three approaches compared with the same 10-trial protocol: **imitation learning** (ACT → SmolVLA + RTC + DAgger, 176 human demos + 40 corrections → 9/10 stack, 7/10 unstack, 5/5 counterfactual), a **classical pipeline** (camera calibration → IK → wrist-camera visual servo → 10/10 in-distribution, 7/10 out-of-distribution vs. 4/10 and 0/10 for the learned policy), and **sim-to-real** (a MuJoCo simulator fitted by measurement, 296 auto-generated demos, zero human demos → 2/10 on the real arm, the same range as the real-demo policies; simulated success did not predict transfer, and the bottleneck was demonstration diversity, not physics). English summary: `research_summary_en.md` · 2-minute video: `videos/summary_2min_1009.mp4` · daily research log (Korean, every failure included): `SO101_연구노트.md`. Repository: https://github.com/GGOMOONG/so101-cube-stacking

---

1. 문제

3 cm 큐브 두 개(주황, 빨강)가 흰 테이블 위에 놓여 있다. 로봇은 주황을 집어 빨강 위에 올려야 한다. 카메라는 테이블 위 약 55 cm의 탑 카메라 1대와 그리퍼에 붙은 손목 카메라 1대.

질문은 세 단계로 커졌다.

1. **모방학습으로 되는가?** (주제 1) 사람 시연을 보고 배운 정책이 쌓기를 할 수 있는가.
2. **한 모델이 문장에 따라 쌓기와 허물기를 구분할 수 있는가?** (주제 2) 언어 조건 정책과 DAgger 교정.
3. **학습 없이 계산만으로 같은 작업을 하면 어디까지 되고, 학습과 어디서 갈리는가?** (주제 3) 고전 파이프라인 구현과 정면 비교.
4. **사람 시연 없이, 시뮬레이터에서 자동 생성한 시연만으로 학습한 정책이 실물에서 되는가?** (주제 4) MuJoCo 시뮬레이터 구축, 시연 자동 생성, 시뮬 전용 정책의 실물 평가.

2. 방법

2.1 모방학습 (주제 1·2)

단계	내용
시연	리더 암으로 텔레오퍼레이션. 왼손잡이용으로 리더를 개조(모터 신호 반전). 총 시연 176회 + DAgger 교정 40회
정책	ACT(2단 쌓기) → SmolVLA + RTC(언어 조건 통합 모델)
언어 조건	"주황을 빨강 위에 놓아라" / "빨강 위의 주황을 테이블에 내려놓아라" 두 문장을 한 모델로
DAgger	정책이 막히는 순간 사람이 이어받아 마무리한 장면을 교정 데이터로 추가(20회 × 2차례)
평가	10회 성공률, 실패 유형 분류, 반사실 시험(이미 쌓여 있으면 아무것도 하지 않아야 함)

2.2 고전 파이프라인 (주제 3)

학습 없이 다음 사슬을 직접 구현했다.

```
체스판 → 탑 카메라 렌즈 보정 (RMS 0.34 px)
ArUco 격자 → 카메라 자세·호모그래피 (재투영 0.45 mm)
HSV + 윗면 분리 → 큐브 mm 위치 (오차 ≤ 3 mm)
URDF 순기구학(직접 구현, MuJoCo와 0.000 mm 일치) + 4점 짚기 → 관절 영점·격자↔베이스 변환 (잔차 1.5 mm)
수치 역기구학(scipy) → 관절 각도
손목 카메라 시각 서보 → 잡기 직전 위치·회전 미세 조정 (0.4~1.5 mm)
FK 폐루프 정착 보정 → 팔의 처짐·백래시 흡수
MuJoCo 간섭 검사 → 손목 회전 후보 중 빨강과 안 부딪히는 것 선택
```

핵심 도구: `calib_intrinsics.py`, `calib_homography.py`, `cube_locate.py`, `so101_fk.py`, `so101_ik.py`, `calib_kinematics.py`, `grid_frame_fit.py`, `grasp_center_calib.py`, `wrist_view.py`, `sim_check.py`, `pick_stack.py`(전체 실행).

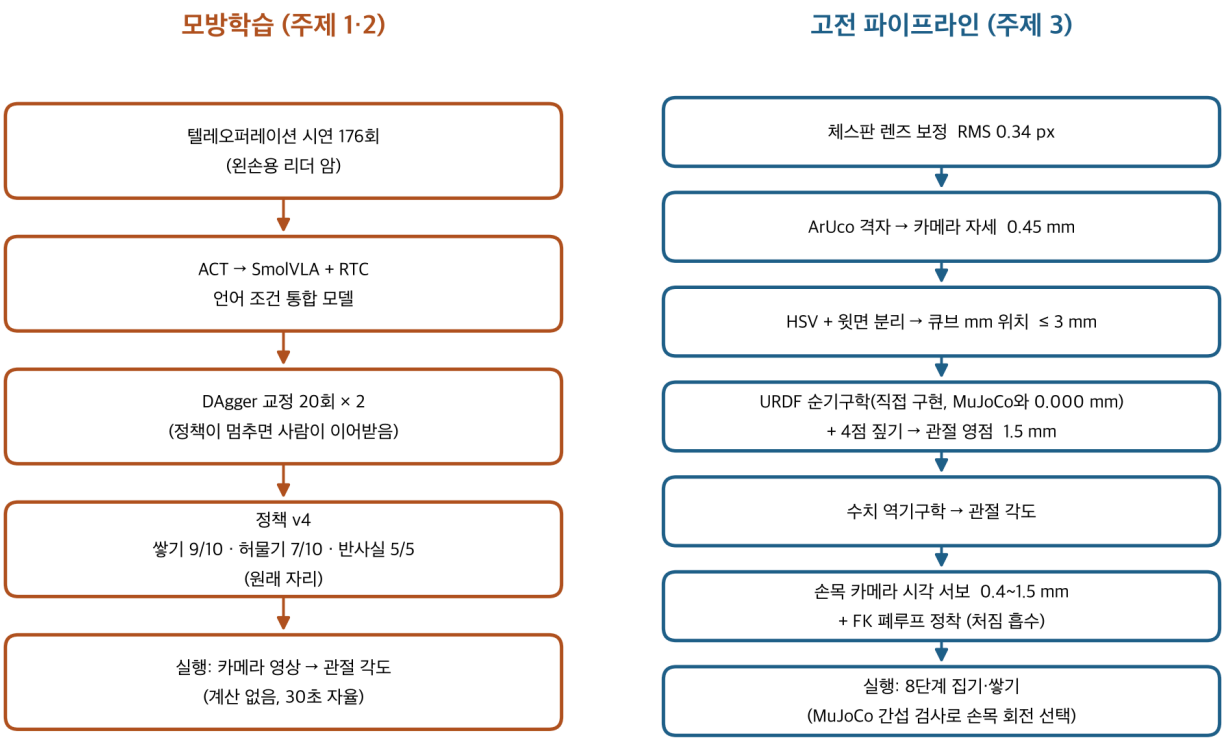


그림 1. 같은 작업을 푸는 두 파이프라인. 왼쪽은 모방학습, 오른쪽은 고전 파이프라인(각 단계의 실측 정확도 포함).

2.3 시뮬레이션→실물 (주제 4)

주제 3에서 만든 보정값·기구학·실물 기준선 위에 MuJoCo 시뮬레이터를 세우고, 사람 시연을 한 번도 쓰지 않은 정책을 실물에서 평가했다.

단계	내용
물리	SO-101 URDF → MJCF. 실물 시연 12회(6,207프레임)의 명령을 재생해 관절 궤적 RMSE로 동역학 맞춤: 명령 지연 3프레임(100 ms), 관절별 위치 게인·감쇠·마찰(검증 RMSE 0.4~1.3°). 큐브 접촉은 MuJoCo 기본값이 너무 물러 턱 판이 5.6 mm 파고들기에 단단한 접촉으로 교체(쥔 상태 그리퍼 읽기 13.5, 실물 13.8)
카메라	탑: 주제 3의 렌즈 보정·호모그래피·왜곡을 그대로 가상 카메라에 적용. 손목: ArUco 30장으로 hand-eye 보정 + 자실측(렌즈–턱 끝 110 mm)으로 검증, 기울기 +3° 보정. 외관은 실물 사진 통계(책상 밝기, 큐브 HSV, 그림자 비율 0.88)에 맞춤, 손목 카메라 기판도 형상으로 추가
시연 생성기	역기구학 계획기가 실물 시연의 통계(접근 방향·속도·시작 자세·그리퍼 명령·회차 길이 16 s)를 따라 320회씩 자동 생성. 배치는 실물 분포 절반 + 도달 범위 전체 절반. 도메인 무작위화(조명·색·크기·카메라 ±2 mm), DART식 교란(실행 명령에 2~2.5° 관절 잡음, 기록은 깨끗한 명령)
학습	SmolVLA base에서 시뮬 데이터만으로 미세조정(10k~30k 스텝, 2.5k~5k마다 체크포인트). 데이터셋 v1(무른 큐브) → v2(단단한 큐브·게인 수정·놓기 보정) → v3(손목 카메라 오보정, 무효) → v3b(v2 + 기판·조명·DART)
평가	① 실물 정책(v4·v5)을 시뮬에서 실행해 실물→시뮬 갭의 요인을 분리, ② 시뮬 정책을 체크포인트마다 시뮬에서 10회 평가(학습 곡선), ③ 시뮬 정책을 실물에서 A/B 10회(주제 3과 같은 조건, P 16, 격자 없음)

핵심 도구: `sim_phys.py`(물리·외관), `sim_cam.py`(가상 카메라·왜곡), `sim_sysid.py`(동역학 맞춤), `sim_demo.py`(시연 생성기), `sim_policy_eval.py`(시뮬 내 정책 평가),

`wrist_cam_calib.py` · `wrist_cam_fit_cube.py` (손목 카메라), `runpod_sim_*.sh` · `watch_ckpt_*.sh` (클라우드 학습·자동 평가).

2.4 비교 조건

- **A. 시연 범위 안 배치**: 주황 왼쪽, 빨강 오른쪽, 간격 8~10 cm (학습 데이터와 같은 배치)
- **B. 시연 범위 밖 배치 10종**: 좌우 바꿈, 대각선, 간격 좁게, 둘 다 멀리, 둘 다 가까이, 앞뒤 일렬, 양끝
- 성공 기준: 주황이 빨강 위에 놓여 3초 이상 유지. 각 조건 10회.
- 고전은 모터 P 게인 48(처짐 5.7 → 2.3 mm), 학습은 P 16(학습 조건. 48에서는 팔이 떨려 동작 불가).

3. 결과

3.1 모방학습의 진행

모델	데이터	쌓기	허물기	반사실	비고
ACT v1	시연 50	1/10	–	–	색이 아니라 화면 위치로 역할을 배움
ACT (증강)	시연 94	8/10	–	–	데이터 구조를 고치자 성공
ACT 5단 쌓기	시연 100	0/10	–	–	그리퍼 여닫기 결정 불능(11~36회/회차)
SmolVLA v2 통합	시연 176	3/10 → 2/10 (학습 2배)	9/10	5/5	빨강 위에서 놓지 못하고 정지 8회. 학습 연장 효과 없음
v3 = v2 + DAgger 20	196	6/10	10/10	5/5	정지 8 → 3회
v4 = v3 + DAgger 20	216	**9/10**	7/10	5/5	쌓기 교정만 넣자 허물기가 내려옴

3.2 고전 대 학습 (같은 날, 같은 자리, 같은 큐브)

조건	고전 파이프라인	학습 정책 v4
A. 시연 범위 안	**10/10**	**4/10**
B. 시연 범위 밖	**7/10**	**0/10**
배경 변화(격자 종이 위)	격자가 필수	0/10
제어 게인 16 → 48	정확도 향상	떨림, 동작 불가

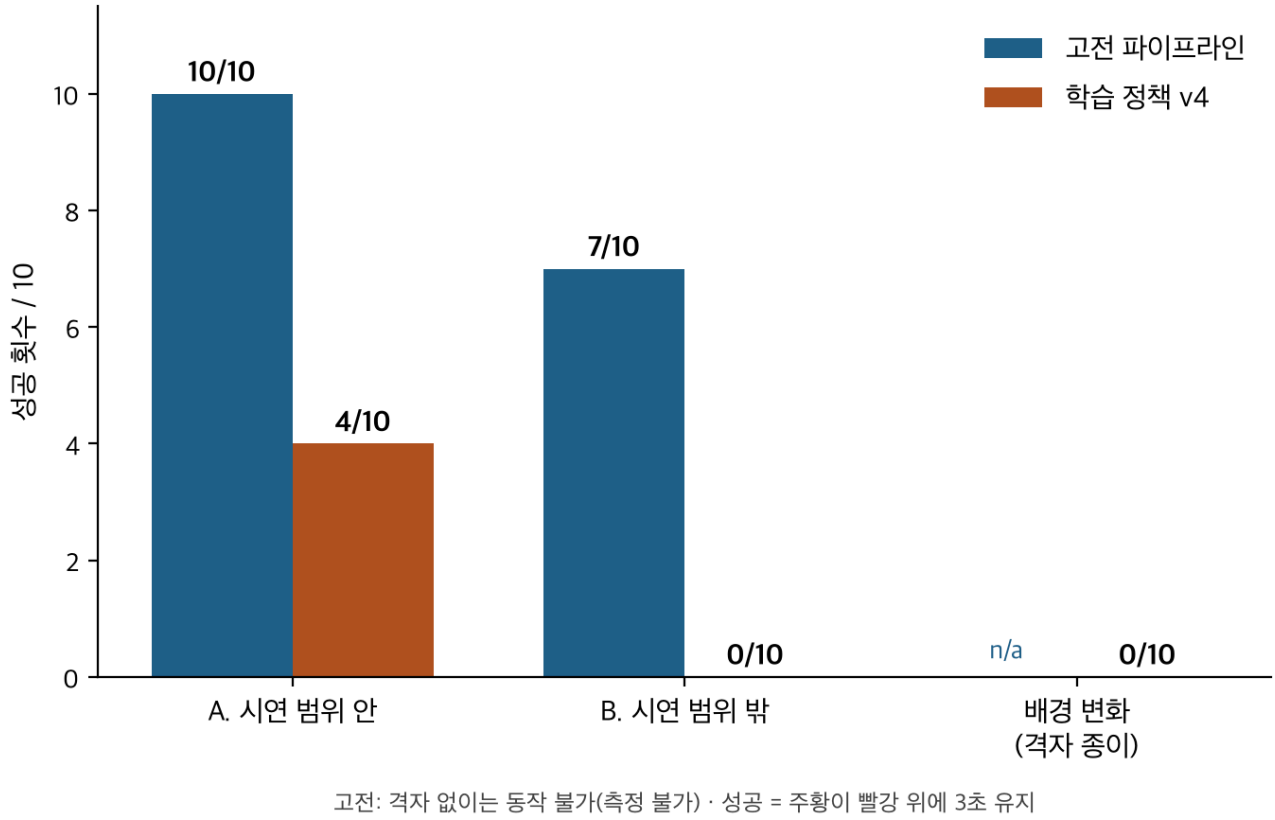


그림 2. 조건별 성공 횟수(각 10회). 고전은 격자 없이는 동작할 수 없어 배경 변화 조건이 없다.

실패 단계별 분류:

단계	고전 (B, 10회)	학습 (A+B, 20회)
인식	0	1 (좌우 바꾸면 인식 못 함)
잡기	0 (20회 전부 잡음, 그중 11회는 손목 카메라로 물림 확인)	5
옮기기	0	0
놓기	3 (위치 10 mm 이상 오차 → 모서리)	10 (빨강 밀림 7, 그중 1회는 모서리 동반 / 모서리 2 / 못 놓음 1)

3.3 시뮬레이션→실물 (주제 4)

실물 정책을 시뮬에 넣어 본 결과(실물→시뮬 갭): v4는 실물 A 4/10인데 맞춤 동역학 시뮬에서는 0/10. 요인을 하나씩 바꿔 가며 A 8회씩 비교하면(잡기 횟수) 기준 3/8 → 명령 지연 0 **8/8**, pan·roll 게인 완화 **8/8**, 그리퍼 게인 ×0.7 4/8, 팔 전체 게인 ×0.5 1/8, 숨은 처짐 추가 0/8, 조명 변경 7/8. 즉 실물 정책이 시뮬에서 막히는 주원인은 **명령 지연과 pan·roll 게인**이고, 처짐·마찰·조명은 부차적이다. 이 과정에서 무른 큐브 접촉 오류(쥔 상태 읽기 3 vs 실물 13.6)를 찾아 v2 이후 데이터에 반영했다.

시뮬 전용 정책의 학습 곡선(시뮬 A 10회)

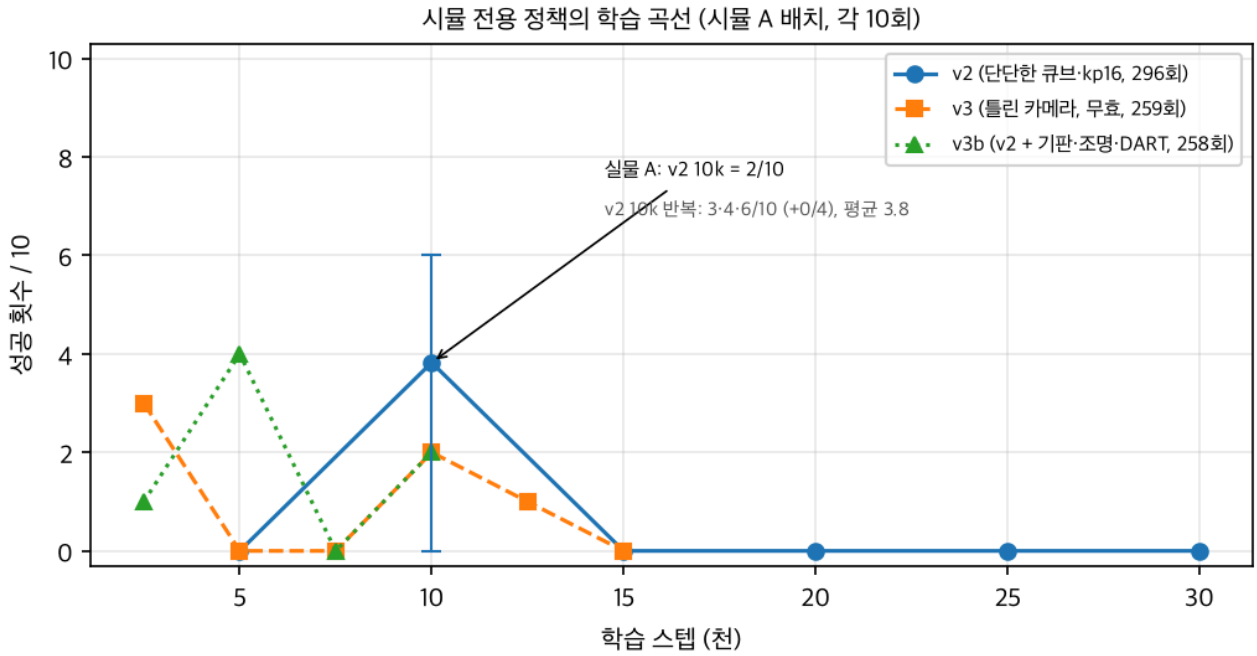


그림 3. 시뮬 데이터만으로 학습한 세 정책의 체크포인트별 시뮬 성공 횟수. 세 번 모두 2.5k~10k가 최고이고 이후 0~2/10로 떨어진다.

데이터셋	시연	최고 체크포인트 (시뮬 A)	최종 (시뮬 A / B)
v1 무른 큐브	310	10k 0/10 (잡기 0)	학습 중단(팟 오류)
v2 단단한 큐브·게인·놓기 보정	296	**10k 3·4·6/10 (반복 3회)** (B 4/10)	30k 0/10 / 2/10
v3 손목 카메라 오보정	259	2.5k 3/10	15k 0/10 / 2/10 (무효)
v3b v2 + 기판·조명·DART	258	5k 4/10	10k 2/10 / 2/10

시뮬 전용 정책의 실물 평가(P 16, 격자 없음, 각 10회)

정책	실물 시연	A. 범위 안	B. 범위 밖	비고
v4 (실물 시연 216)	216	4/10	0/10	주제 3 기준선
v5 = v4 + DAgger 3차 (236)	236	3/10	1/10	3차 교정 효과 없음
시뮬 v2 10k	**0**	**2/10**	**0/10**	A: 실물 시연 없이 쌓기 완수. 잡기 실패 5(주황 밀어냄), 놓기 실패 3. B: 인식 1·놓기 1·잡기 8 실패, v4 B와 같은 양상
시뮬 v3 2.5k (오보정 카메라)	0	0/10	–	벌어지는 턱이 주황을 눌러 밀어냄 → 카메라 오류의 증거
시뮬 v3b 10k	0	0/10	–	전부 잡기 실패(못 잡음 7, 밀어냄·흐트러뜨림 3)
시뮬 v3b 5k (시뮬 최고점 4/10)	0	0/10	–	전부 잡기 실패. v3b는 두 체크포인트 합쳐 0/20 → 데이터셋 자체가 전이를 해침

3.4 수치의 통계적 해석

모든 성공률은 조건당 10회 측정이다. 10회 중 k회 성공의 95 % 신뢰구간(Clopper–Pearson)은 0/10 → 0~31 %, 2/10 → 3~56 %, 4/10 → 12~74 %, 7/10 → 35~93 %, 9/10 → 55~100 %, 10/10 → 69~100 %다. 따라서 이

문서의 주장 가운데 **유지되는 것**은 구간이 겹치지 않거나 반복으로 뒷받침된 것들이다: 고전 A 10/10 대 학습 A 4/10, 고전 B 7/10 대 학습 B 0/10, 언어 조건 반사실 15/15 무동작, DAgger 전후 쌓기 2/10 → 9/10, 시뮬 v3b의 실물 0/20 대 시뮬 v2의 2/10(같은 조건, 0/20의 상한 17 %). **유지되지 않는 것**은 v4 4/10 대 v5 3/10, 시뮬 v2 실물 2/10 대 실물 정책 3~4/10, 시뮬 내 체크포인트 간 1~2회 차이다. 이들은 "같은 구간"이라고만 말할 수 있다. 시뮬 v2 10k를 같은 조건에서 세 번 반복하면 3·4·6/10으로, 10회 측정의 흩어짐은 ±2회 정도다.

4. 실패 분석

학습 정책이 무너지는 곳

- 옛 자리 9/10 → 새 자리 4/10 → 격자 종이 위 0/10 → 범위 밖 배치 0/10. 가중치는 같은데 "보는 화면"이 달라질수록 떨어진다. 좌우를 바꾸면 인식 자체를 못 하고, 멀리·가까이 배치에서는 잡지 못한다.
- 모터 게인을 올리자 팔이 떨렸다. 정책은 시연 때 팔의 느린 반응까지 함께 외웠다. **정책은 작업뿐 아니라 하드웨어의 제어 동역학까지 암묵적으로 학습한다.**
- 실패의 절반 이상은 "빨강 밀림": 놓기 높이를 낮게 잡아 들고 있는 주황으로 빨강을 밀어낸다.

고전 파이프라인이 무너지는 곳

- 잡기는 20/20(11회는 손목 카메라로 물림 확인). 실패는 전부 **놓기 위치**다. 잡기 서보가 찾은 보정을 놓기 자세로 옮길 때 잡기·놓기 자세 차이가 큰 배치에서 10 mm 이상 어긋난다. 놓기에는 폐루프가 없다.
- 팔의 물리: 명령 위치보다 5~15 mm 처짐(P 16), 그리퍼를 여닫을 때 팔이 2~8 mm 내려앉음, 관절 보간이 접힌 자세에서 호를 그려 큐브를 스침. 각각 게인 상향, 정착 보정, 직선 경로로 대응했다.
- **"URDF가 틀렸다"는 처음 진단은 절반만 맞았다.** 잡기 실패 7회 뒤 실측으로 잡기 중심 13/7 mm·턱 축 약 40° 어긋남을 찾아 보정하고 손목 카메라 서보로 첫 성공을 얻었다. 시뮬레이터를 만들며 재검토하니(10-01) 40°는 URDF가 아니라 09-23 보정의 손목 롤 영점 오차(약 36°), 13 mm는 한쪽 턱만 움직이는 그리퍼 기하였다. "20 mm 안쪽 패드"는 실물에 없다. 보정값이 현상을 정확히 덮어 실물은 잘 동작했지만 원인은 잘못 짚었었다.

시뮬 전용 정책이 무너지는 곳

- **일찍 과적합한다.** 세 데이터셋 모두 2.5k~10k 체크포인트가 최고(약 4/10; v2 10k 반복 3·4·6)이고 그 뒤 0~2/10(같은 체크포인트 10회 반복의 흩어짐은 ±2 정도). 후반 체크포인트는 잡기는 되는데 놓기 위치를 일정하게 2 cm 비켜 놓는다(외운 궤적). 계획기 시연이 항상 같은 방식으로 성공만 하므로, "보고 고치는" 행동을 배울 재료가 없다. DART식 교란(v3b)은 이 천장을 올리지 못했다.
- **시뮬 내 성공률은 실물 전이를 예측하지 못한다.** v3b는 시뮬에서 v2와 같은 수준(최고 4/10)이었지만 실물에서는 두 체크포인트 모두 0/10(합계 0/20). v2에 더한 세 가지(기판 형상·그림자 완화 조명·DART 교란) 중 무엇이 해쳤는지는 분리하지 못했다(각각 따로 학습해야 함). DART는 실행 궤적은 교란되고 기록 상태는 깨끗한 명령이라 화면–상태 관계가 실물과 달라지는 점이 의심된다.
- **실물 실패 양상은 실물 정책과 같다.** 시뮬 v2의 실물 실패는 "주황을 밀어냄"(잡기) 5회와 놓기 3회로, v4·v5의 실패 유형과 겹친다. 범위 밖 배치(B)에서는 0/10으로, 시뮬 안에서 B 4/10이던 일반화가 실물로 옮겨지지 않았다(v4 B 0/10과 같음). 시뮬에서 배운 "손목을 90° 돌려 잡기"(계획기가 간섭을 피하려 고른 회전)가 실물에서도 그대로 나왔다.

- **카메라 모델 오류 한 번이 데이터셋 하나를 날렸다.** 손목 사진과 시뮬 렌더의 큐브 실루엣을 맞출 때 큐브를 고정 턱 면 선 위에 놓는 실수(실제 중심은 열림 방향으로 15 mm)로 카메라를 9 mm 잘못 옮겼고, 그 카메라로 만든 v3는 실물 0/10이었다. 실물에서 "벌어지는 턱이 주황을 누른다"는 관찰이 오류를 드러냈고, 바로잡은 뒤 원래 hand-eye 카메라가 맞다는 것(IoU 0.61 vs 0.35)을 확인했다. 자로 잰 110/65/90 mm가 기준점이 됐다.

4.1 기존 연구와의 대조

- **SmolVLA**(Shukor 외 2025)는 커뮤니티가 모은 저가 로봇 데이터로 사전학습하고 비동기 추론으로 소비자 하드웨어에서 실시간 제어를 한다고 보고한다. 이 프로젝트의 결과는 그 주장과 맞다: RTC 없이 0/10, RTC로 6/10. 다만 논문의 실물 평가는 시연을 수집한 환경에서 이뤄지므로, 자리를 옮겼을 때 9/10 → 4/10으로 떨어지는 "환경 가정" 효과는 논문 수치에 드러나지 않는다.
- **DART**(Laskey 외 2017)는 시연자의 정책에 잡음을 주입해 복구 행동을 시연에 담고, 어지러운 환경에서 잡기 성능이 행동 복제 대비 평균 62 % 올랐다고 보고한다. 이 프로젝트의 v3b는 같은 원리를 계획기에 적용했지만(실행 명령에 2° 잡음, 기록은 깨끗한 명령) 실물 전이가 0/20으로 나빠졌다. 차이는 셋이다. DART의 시연자는 사람이고 잡음 수준을 정책 오차에 맞춰 최적화하는데, 여기서는 고정 2°였다. DART는 같은 환경에서 평가했지만 여기서는 시뮬→실물 전이를 평가했다. 그리고 v3b는 잡음 외에 외관 변경 두 가지가 함께 들어가 요인이 분리되지 않았다. 따라서 "DART가 틀렸다"가 아니라 "이 적용 방식으로는 전이에 도움이 되지 않았다"가 정확한 서술이다.
- **도메인 무작위화**(Tobin 외 2017)는 비현실적 무작위 질감으로 렌더링한 시뮬 이미지만으로 실물 물체 위치를 1.5 cm 정확도로 추정하는 검출기를 학습했다. 이 프로젝트는 반대로 실물 외관에 맞추고 그 주변에서 약하게 흔드는 쪽을 택했다(조명 ×0.5~1.6, 색 ±5 %, 카메라 ±2 mm). 시뮬 정책이 실물에서 2/10을 낸 것은 이 약한 무작위화로도 전이가 일부 된다는 뜻이지만, 범위 밖 배치 0/10은 무작위화 폭이 좁았음을 시사한다. 질감 수준의 강한 무작위화는 시도하지 않았다.

공통: 두 방식 모두 놓기가 약하다. 고전은 "위치를 계산하고 잡기는 서보로 닫는" 구조가 잘 됐으니, 놓기에도 같은 구조(놓기 서보)를 넣거나, 접근·정렬은 고전으로 하고 잡기·놓기는 학습으로 하는 혼합 구조가 다음 단계다.

5. 배운 것

1. **각 방식은 자기 환경 가정에 묶여 있다.** 고전은 격자·보정·게인이라는 눈에 보이는 가정이라 고칠 수 있었고(범위 밖 예비 0/8 → 코드 수정 4건 뒤 7/10), 학습은 시연 때의 배경·배치·팔 반응이라는 보이지 않는 가정이라 데이터를 다시 모아야 한다.
2. **데이터 구조가 학습량보다 중요하다.** ACT v1의 1/10은 시연이 "두 번 집기"였기 때문이고, 통합 모델 v2는 학습을 두 배(3만 → 6만 스텝)로 늘려도 3/10 → 2/10으로 나아지지 않았다. 5단 쌓기는 시연 100회·10만 스텝에도 0/10이었다.
3. **DAgger는 효과가 있지만 균형이 필요하다.** 쌓기 교정 40회로 쌓기 2 → 9/10, 대신 허물기 10 → 7/10.
4. **하드웨어 진단은 부품 맞바꾸기가 이론보다 빠르다.** 일주일 끌던 손목 카메라 정지는 잡음(페라이트 코어)이 아니라 케이블이었고, 케이블 한 번 바꿔 확인했다.
5. **시뮬레이터는 물리를 켜지 않으면 불가능한 자세도 보여 준다.** MuJoCo로 FK를 검증하고 궤적·간섭을 미리 봤지만, 시작 자세를 임의로 넣었더니 팔꿈치가 테이블 아래에 있었다. 실측값을 써야 한다.

6. **기록이 실험을 구했다.** 매일 남긴 연구노트 덕에 "카메라가 30 cm 움직였다"(실은 해상도 차이), "턱이 헛돈다"(실은 측정 지점 차이) 같은 오판을 되돌릴 수 있었다.
7. **시뮬 데이터만으로도 실물 시연 정책과 같은 구간에 닿는다.** 사람 시연 0회인 시뮬 v2가 실물 2/10, 실물 시연 216~236회인 v4·v5가 같은 환경에서 3~4/10. 무료로 만든 데이터가 실물 수준에 근접했다는 뜻이지, 넘어섰다는 뜻은 아니다(n=10, 차이는 잡음 범위).
8. **시뮬의 병목은 물리나 카메라가 아니라 시연의 다양성이다.** 동역학·접촉·카메라·조명을 실측으로 맞춘 뒤에도 시뮬 내 천장은 약 4/10(반복 3·4·6)이고 더 학습하면 떨어진다. 계획기가 만든 시연은 사람 시연보다 균일해서 정책이 외워 버린다.
9. **맞춤(fitting)은 가정이 틀리면 자신 있게 틀린 답을 준다.** 카메라 맞춤은 IoU가 올라갔다고 보고했지만 큐브 위치 가정이 틀려 있었다. 실물의 실패 양상과 자 실측이 오류를 잡았다. 최적화 결과는 독립된 측정으로 한 번은 대조해야 한다.

6. 다음

- **놓기 서보**: 놓기 직전 탑·손목 카메라로 들고 있는 큐브와 목표를 비교해 보정. 현재는 그리퍼 가림으로 측정 치우침 12~27 mm.
- **혼합 구조**: 접근·정렬은 고전, 잡기·놓기는 학습.
- **시뮬 시연의 다양화**: 계획기에 접근 전략·속도·실패 후 재시도를 섞고, 관측(화면 질감·노출)에도 무작위화를 넣어 과적합 천장(약 4/10)을 올리는 것이 시뮬→실물의 다음 단계. 실물 시연 소량 + 시뮬 시연 혼합 학습은 그다음.
- **손목 카메라 커넥터 고정**: 손목 움직임에 커넥터가 당겨져 간헐 정지가 재발. 실패 원인 분리를 위해 먼저 해결할 하드웨어 항목.
- **양팔**: 두 번째 SO-101과 카메라 1대 추가, 격자 보정 재사용.

7. 저장소 구성

```
robotics/
├── README.md                        이 문서
├── SO101_연구노트.md                일자별 연구노트(전 과정, 실패 포함)
├── pick_stack.py                    고전 파이프라인 전체 실행 (서보·정착·물림 확인·자동 모드·결과 CSV)
├── calib_*.py, cube_locate.py       카메라 보정·큐브 위치
├── so101_fk.py, so101_ik.py         기구학(URDF 직접 구현) / 역기구학
├── calib_kinematics.py, grid_frame_fit.py, touch_points.py   관절 영점·격자 변환
├── grasp_center_calib.py, gripper_geom.py, wrist_view.py     그리퍼 실측·손목 카메라 서보
├── sim_view.py, sim_plan.py, sim_check.py, sim/               MuJoCo 뷰어·궤적 미리보기·간섭 검사
├── sim_phys.py, sim_cam.py, sim_sysid.py, sim_demo.py         시뮬 물리·가상 카메라·동역학 맞춤·시연 생성기
├── sim_policy_eval.py, watch_ckpt_*.sh, runpod_sim_*.sh       시뮬 내 정책 평가·체크포인트 자동 평가·클라우드 학습
├── wrist_cam_calib.py, wrist_cam_fit_cube.py, wrist_pose_capture.py   손목 카메라 hand-eye·실루엣 검증
├── sag_test.py, grip_test.py, goto_grid.py, read_joints.py    하드웨어 시험
```

```
├── v4_eval.sh, loop_stack_unstack.sh                          학습 정책 평가·루프 시연
├── calib/                          보정 결과, 실행 사진, 결과 CSV(classical_runs.csv,
v4_runs.csv, sim_policy_runs.csv, sim_dynamics.json)
├── photos/, videos/                환경 사진, 시연 영상
└── so101_model/                    SO-101 URDF + 메시
```

7.1 실행 환경

- macOS (Mac mini M4 Pro), Python 3.12, conda 환경 `lerobot`. 주요 버전: lerobot 0.6.2(편집 설치), torch 2.11, mujoco 3.14, opencv 4.13, numpy 2.2, scipy 1.18.
- 로봇: SO-101 팔로워 `/dev/tty.usbmodem…`, 카메라 인덱스 0 = 탑, 1 = 손목(macOS 재부팅 시 바뀔 수 있음 → `cam_check.py`로 확인). 리더 암은 왼손용으로 그리퍼 신호 반전.
- 학습은 RunPod(스크립트 `runpod_*.sh`, 허브 토큰은 팟에서 `hf auth login`), 평가는 Mac(mps)에서 실행.
- 보정값(`calib/*.json`)은 이 하드웨어 한 대에 묶여 있으므로 다른 팔에서는 `calib_*.py` 순서대로 다시 잰다.
- `so101_model/`의 URDF·메시는 TheRobotStudio SO-ARM100 저장소(Apache 2.0)에서 가져와 손목 롤 영점 등을 보정한 것.
- 설치 요약: `conda create -n lerobot python=3.12` → LeRobot 편집 설치 (`pip install -e ".[smolvla]"`) → `pip install mujoco opencv-python scipy markdown pymupdf`.

7.2 데이터셋·모델 (허깅페이스)

용도	저장소	공개
실물 시연 + DAgger 교정(v4 학습 데이터, 216회)	`moonbeards/color_cube_stack_unstack2_dagger2`	공개
시뮬 시연 v2(296회, 시뮬→실물 전이에 쓴 데이터)	`moonbeards/sim_stack_v2`	공개
실물 시연 정책 v4 (SmolVLA)	`moonbeards/smolvla_color_cube_stack_unstack2_v4`	공개
시뮬 전용 정책 v2 10k (SmolVLA)	`moonbeards/smolvla_sim_stack_v2` (체크포인트 `checkpoint_010000` 사용)	공개
중간 단계·롤아웃·무효 데이터 (`sim_stack_v1/v3/v3b` 등), 나머지 모델	비공개	문의 시

8. 참고문헌

1. Zhao, T. Z., Kumar, V., Levine, S., Finn, C. (2023). *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT). RSS 2023. arXiv:2304.13705
2. Shukor, M. 외 (2025). *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics*. arXiv:2506.01844

3. Black, K., Galliker, M., Levine, S. (2025). *Real-Time Execution of Action Chunking Flow Policies* (RTC). arXiv:2506.07339
4. Ross, S., Gordon, G., Bagnell, J. A. (2011). *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning* (DAgger). AISTATS 2011
5. Cadene, R. 외 (2024). *LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch*. github.com/huggingface/lerobot
6. TheRobotStudio. *SO-ARM100 / SO-101*. github.com/TheRobotStudio/SO-ARM100
7. Todorov, E., Erez, T., Tassa, Y. (2012). *MuJoCo: A physics engine for model-based control*. IROS 2012
8. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools
9. Laskey, M., Lee, J., Fox, R., Dragan, A., Goldberg, K. (2017). *DART: Noise Injection for Robust Imitation Learning*. CoRL 2017. arXiv:1703.09327
10. Tobin, J. 외 (2017). *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. IROS 2017. arXiv:1703.06907

9. 작성 방식에 관한 고지

이 프로젝트는 AI 코딩 도구(Claude)와 함께 진행했다. 역할은 다음과 같다. **본인**: 연구 질문과 주제 전환 결정, 하드웨어 조립·개조·진단, 모든 시연 수집과 실물 실험의 수행·관찰·성공 판정, 각 단계에서 무엇을 다음에 할지에 대한 결정, 결과 해석의 최종 판단. **AI 도구**: 보정·기구학·시뮬레이터·시연 생성기·평가 도구의 코드 작성 대부분, 분석 스크립트와 그림, 문서 초안, 그리고 본인이 승인한 범위 안에서 로봇이 필요 없는 시뮬 평가의 자율 실행(예: 10-01~02 야간). 실험 설계는 AI가 제안하고 본인이 채택·수정·기각하는 방식으로 했다. 연구노트에는 이 과정이 날짜별로 그대로 남아 있고, AI의 오류(10-01 손목 카메라 맞춤의 가정 오류)와 그것을 실물 관찰로 바로잡은 기록도 포함된다.